Association Rule Mining and Neural Networks for Smart Retail Recommendations

Project report submitted to Christ College (Autonomous) in partial fulfilment of the requirement for the award of M.Sc. Degree programme in Statistics

by

C R ASWATHY

Register No.CCAYMST005

2026

Certificate

This is to certify that the project entitled 'Association Rule Mining and Neural Networks for Smart Retail Recommendations', submitted to the Department of Statistics in Partial fulfillment of the requirements for the award of the Masters Degree in Statistics, is a bonafied record of original research work done by C R ASWATHY (CCAYMST005) during the period of his study in the Department of Statistics, Christ College (Autonomous) Irinjalakuda, Thrissur, under my supervision and guidance during the year 2024-2026.

Ms. Jiji M B
Assistant Professor
Department of Statistics
Christ College (Autonomous)
Irinjalakuda

Dr.Davis Antony M
Head, Department of Statistics
(Self Financing)
Christ College(Autonomous)
Irinjalakuda

External Examiner:

Place:
Date:

Declaration

I hereby declare that the matter embodied in the project entitled 'Association Rule Mining and Neural Networks for Smart Retail Recommendations ', submitted to the Department of Statistics in partial fulfillment of the requirements for the award of the Masters Degree in Statistics, is the result of my studies and this project has been composed by me under the Guidance and Supervision of Jiji M B, Assistant Professor, Department of Statistics, Christ College (Autonomous) Irinjalakuda, during 2024-2026.

I also declare that this project has not been previously formed the basis for the award of any degree, diploma, associate ship, fellowship etc. of any other university or institution.

Irinjalakuda

Date: **C R ASWATHY**

Acknowledgement

This project would not have been possible without the guidance and the help of several individuals who in one way or another contributed and extended their valuable assistance in the preparation and completion of the study.

I would like to express my deepest gratitude to my guide Jiji M B, Assistant Professor, Department of Statistics,whose guidance has been of immense help in successfully completing this report.

With great pleasure, I take this opportunity to express my sincere gratitude to my respected teachers, friends and non teaching staff of Department of Statistics, Christ College (Autonomous) Irinjalakuda, for their valuable advice and encouragement throughout my course.

Last but not least, I am indebted to my parents for their unconditional love and support.

Irinjalakuda
Date: **C R ASWATHY**

Contents

List of Figures

List of Tables

Abstract

In today's retail environment, a large amount of transaction data is generated every day. Analyzing this data is very important for understanding customer purchasing behavior and improving business strategies. One of the most commonly used techniques for this purpose is Market Basket Analysis, which helps in identifying products that are frequently purchased together. Traditional association rule mining methods such as ECLAT and FP-Growth are widely used to find relationships between items in transactional data. However, these methods mainly identify static patterns and are not very effective in predicting future customer purchases. To overcome this limitation, this project use a hybrid approach that combines association rule mining with deep learning techniques.

In this project, the Online Retail dataset is used for analysis. The dataset is first preprocessed by handling missing values, removing duplicate records, converting data types, and transforming the data into transaction basket format. After preprocessing, association rule mining algorithms such as ECLAT and FP-Growth are applied to generate frequent itemsets and association rules using measures like support, confidence, and lift. In addition, deep learning models such as Convolutional Neural Network (CNN), Bidirectional Long Short-Term Memory (Bi-LSTM), and a hybrid CNN-BiLSTM model are implemented to predict customer purchasing behavior and improve recommendation accuracy.

The results show that the FP-Growth algorithm generates stronger association rules compared to the ECLAT algorithm. Among the deep learning models, the CNN-BiLSTM model achieved the highest prediction accuracy. This study concludes that combining association rule mining and deep learning techniques helps in building an effective retail recommendation system and supports better decision-making in retail businesses.

Chapter 1

INTRODUCTION

1.1 Background

The retail landscape has transformed dramatically in recent years. Stores have way more products than they used to, and what customers want is always shifting. Because there is so much data coming in from every sale, it is becoming really hard for stores to actually see the patterns in what people are buying.

For a long time, stores have used something called Market Basket Analysis (MBA). You have probably seen this in action it is the logic behind 'people who bought this also bought that.' To do this, they usually use classic tools like ECLAT or FP-Growth. These are great at finding simple pairs like bread and butter, but they have a big weakness: they do not really understand the order or the timing of when people buy things. They only look at what is in the basket right now, not what the customer might need next week.

As shopping data gets bigger and more complicated, we need smarter tools. That is why this study brings in Deep Learning. CNNs are used to find quick, local patterns and Bi-LSTMs to look at the 'big picture' of a customer's history. By combining these into one hybrid model, we can actually use the old-school association rules to help the deep learning models focus on the most important data.

To make sure this actually works, this study tests it using two different methods: the classic statistics like Support and Lift, and deep learning metrics like Accuracy. The goal is to build a system that does not just look at the past, but actually predicts what a customer will want next. For a retailer, having that kind of insight is a huge advantage in a competitive market.

1.2 Problem Statement

Retailers generate massive amounts of transactional data every day, but traditional market basket analysis methods like ECLAT and FP-Growth only reveal static associa-

tions between products. They fail to capture the order and deeper context of customer purchases, making it difficult to predict what shoppers will buy next. To overcome this limitation, there is a need for a hybrid approach that combines association rule mining with deep learning models, enabling retailers to move from simply identifying frequent itemsets to accurately forecasting future buying behavior.

1.3 Objectives

- To apply association rule mining Algorithms(Eclat ,FP-Growth)
- To implement deep learning models (CNN,BiLSTM,CNN-BiLSTM)
- To evaluate and compare model performance

1.4 Scope of the Project

The scope of this project is to replicate and validate a hybrid framework for market basket analysis that combines association rule mining with deep learning models. It involves applying algorithms such as ECLAT and FP-Growth to extract frequent itemsets from retail transaction datasets, and then using CNN, Bi-LSTM, and CNN-BiLSTM architectures to classify and predict customer purchase behavior. The project is limited to analyzing transactional data from selected datasets, implementing the models as described in prior research, and evaluating their performance using standard metrics like support, confidence, lift, accuracy, and loss. By following this framework, the project aims to demonstrate how integrating rule-based insights with predictive modeling can enhance recommendation systems and support smarter decision-making in the retail sector.

Chapter 2

LITERATURE REVIEW

In the past, many researchers conducted Market Basket Analysis (MBA) using various data mining techniques. A brief review of previous research related to MBA using association rules and deep learning methods on offline and online retail datasets is delineated as follows.

The FP-Growth algorithm [10] mines frequent patterns without candidate generation by compressing the database into an FP-Tree structure, significantly reducing computational complexity and memory usage compared to traditional methods. Shabtay et al. [17] extended the FP-Growth algorithm for mining multitude-targeted itemsets and class association rules in imbalanced data, demonstrating its adaptability to complex real-world scenarios. The ECLAT algorithm [19] uses a vertical data format with tid-lists for efficient intersection operations. Zaki and Gouda [20] further improved this approach using diffsets for fast vertical mining, enhancing the algorithm's performance for large-scale transactional datasets.

Recent research has focused on applying deep learning methods to retail analytics. Alzubaidi et al. [2] provided a comprehensive review of CNN architectures, challenges, and applications across multiple domains. LeCun, Bengio, and Hinton [14] discussed the fundamental concepts of deep learning and its transformative impact on various industries. For sequential data processing, Hochreiter and Schmidhuber [11] introduced Long Short-Term Memory (LSTM) networks, which effectively capture long-term dependencies in sequential data. Graves and Schmidhuber [7] developed Bidirectional LSTM (Bi-LSTM) for capturing context from both forward and backward directions, making it particularly suitable for sequence prediction tasks. Liu et al. [16] proposed a hybrid CNN-LSTM model that combines local feature extraction with sequential context understanding, achieving improved performance on text classification tasks.

Market basket analysis has been extensively studied using association rules. Kaur and Kang [12] demonstrated how MBA identifies changing trends in market data, helping retailers understand evolving customer preferences. Unvan [18] applied association rules for market basket analysis using the FP-Growth algorithm, generating top ten rules based on conviction values. Guha et al. [8] examined how artificial intelligence

and deep learning will affect the future of retailing, highlighting the potential of these technologies for personalized recommendations, inventory management, and customer experience enhancement.

Despite the widespread use of association rule mining for identifying customer purchase patterns, the integration of association rules with deep learning methods remains limited. Ghous, Malik, and Rehman [5] addressed this gap by proposing a framework that uses association rules as feature selection for deep learning methods. Their experiments on the InstaCart dataset showed that Bi-LSTM achieved 0.92 accuracy, while on the Bites Bakers dataset, CNN-BiLSTM performed best with 0.77 accuracy.

After reviewing previous research, several observations can be made. Many authors conducted MBA using association rules to find customer purchase habits. MBA helps retailers identify keystone products that are recognized in the market. However, as data continues to increase due to growing customer demands, it becomes difficult to understand customer purchase behavior by only extracting association rules. Association rules alone are only used to find customer purchase behavior, not to predict future purchases. Therefore, association rules and deep learning methods need to be combined to predict and classify these rules. There is a lack of using association rules as feature selection with deep learning methods [5]. This study addresses this gap by using association rules extracted from the Online Retail dataset [4] as input features for CNN, Bi-LSTM, and CNN-BiLSTM models to predict the next product category that a customer is likely to purchase.

Chapter 3

METHODOLOGY

3.1 Data Source and Description

The data comes from the Online Retail dataset, publicly available on the UCI Machine Learning Repository. It contains 5,41,909 transactions with eight attributes: Invoice Number, Stock Code, Description, Quantity, Invoice Date, Unit Price, Customer ID, and Country transaction records from a UK-based online gift shop covering December 2010 to December 2011. The dataset includes 4,372 customers, 3,956 products, and 38 countries. After cleaning, 5,24,878 records were used for analysis. This data is ideal for discovering product associations and predicting customer purchases. Citation: Dua, D., Graff, C. (2019). UCI Machine Learning Repository [Online Retail Dataset].

	InvoiceNo	StockCode	Description	Quantity	\
0	536365	85123A	WHITE HANGING HEART T-LIGHT HOLDER	6	
1	536365	71053	WHITE METAL LANTERN	6	
2	536365	84406B	CREAM CUPID HEARTS COAT HANGER	8	
3	536365	84029G	KNITTED UNION FLAG HOT WATER BOTTLE	6	
4	536365	84029E	RED WOOLLY HOTTIE WHITE HEART.	6	

	InvoiceDate	UnitPrice	CustomerID	Country
0	2010-12-01 08:26:00	2.55	17850.0	United Kingdom
1	2010-12-01 08:26:00	3.39	17850.0	United Kingdom
2	2010-12-01 08:26:00	2.75	17850.0	United Kingdom
3	2010-12-01 08:26:00	3.39	17850.0	United Kingdom
4	2010-12-01 08:26:00	3.39	17850.0	United Kingdom

Figure 3.1: Online Retaile Transactional dataset

3.2 Data Preprocessing

Before applying any analytical algorithms, the raw dataset underwent systematic preprocessing to ensure data quality and suitability for association rule mining. The preprocessing phase was divided into two main stages: data cleaning and data transformation.

3.2.1 Initial Data Cleaning

Data cleaning focused on removing erroneous, irrelevant, and inconsistent records from the dataset.

Removing Duplicate Records Duplicate records occur when the same transaction is recorded multiple times due to system errors. All identical rows were identified and removed using the `drop_duplicates()` function. A total of 5,268 duplicate records were eliminated from the dataset.

Removing Cancelled Transactions Cancelled orders were identified by invoice numbers starting with the letter 'C'. These records do not represent actual completed purchases and were therefore removed. This step eliminated 2,438 cancelled transactions from the dataset.

Removing Returns Returns were identified by negative values in the Quantity column, as returned items are recorded with negative quantities. These records were removed to focus exclusively on positive purchase behavior. A total of 8,923 returns were eliminated.

Removing Invalid Prices Records with zero or negative unit prices were considered invalid, as real products cannot have zero or negative prices. These records were removed to prevent errors in subsequent calculations. This step removed 1,247 records with invalid prices.

Handling Missing Descriptions Records with missing product descriptions were removed because the product identity could not be determined without a description. This ensured that all remaining records had valid product information.

Product Name Standardization Product descriptions were standardized to ensure consistent identification of the same product across different records. The standardization process involved converting all text to uppercase, removing extra spaces, eliminating special characters and punctuation, and replacing common abbreviations with standardized forms. For example, "T-LIGHT" was replaced with "TEA LIGHT", "HOLDER" with "HLDR", and color names like "WHITE", "BLACK", "RED" were shortened to "WHT", "BLK", "RD" respectively. This standardization ensured that

identical products were not treated as different items due to minor variations in their descriptions.

3.2.2 Data Preparation for Association Rule Mining

After cleaning the data, it was transformed into the specific format required for association rule mining algorithms.

Filtering Frequent Products To reduce computational complexity and focus on meaningful patterns, only products that appeared in at least 30 transactions were retained. This filtering step reduced the number of unique products from 3,922 to 2,583, eliminating rare products that would not generate reliable association rules.

Creating Transaction Baskets Transaction baskets were created by grouping all products purchased under the same invoice number. Each basket represents a single transaction and contains a list of all products bought together. This format is essential for association rule mining algorithms such as ECLAT and FP-Growth.

Basket Cleaning After basket creation, two additional cleaning steps were performed. First, duplicate items within each basket were removed to ensure each product appeared only once per transaction. Second, baskets containing only a single item were removed, as no association rules can be generated from such transactions.The final dataset consisted of 18,216 transaction baskets ready for association rule mining.

3.2.3 Preprocessing for Deep Learning Models

Deep learning models require sequence-based representation with fixed-length inputs, preserving the temporal order of purchases.

Chronological Sorting

Transactions were sorted by InvoiceDate in ascending order for each CustomerID to create chronologically ordered sequences of product interactions.

Sequence Construction

For each customer, purchase histories were concatenated into ordered sequences of product IDs. This preserved the temporal patterns in user behavior, unlike association rule mining which treats transactions as unordered baskets.

Integer Encoding

A tokenizer was fitted on all unique product names to map each product to a unique integer index. This encoding transformed categorical product names into numerical values suitable for neural network input. The vocabulary size was determined as the total number of distinct products (3,548). A special token (0) was reserved for padding and unknown products.

Fixed-Length Sequence Padding

All sequences were standardized to a fixed length of 10 products. This length was determined based on the distribution of sequence lengths in the dataset. Sequences shorter than 10 products were padded with zeros at the beginning (pre-padding) to maintain chronological order, where recent products appear at the end of the sequence. Sequences longer than 10 products were truncated by keeping only the most recent 10 products, as recent behavior is typically more predictive of the next purchase.

Target Variable Creation

For each input sequence of fixed length 10, the target variable was defined as the next product ID immediately following the sequence. This created a supervised learning formulation where the model learns to predict the subsequent product based on the preceding 10 products. The target labels were kept as sparse integers, enabling the use of sparse categorical cross-entropy loss.

The length of 10 was selected because the 90th percentile of customers made 10 or fewer purchases, covering 90% of all customers without padding. This length optimally balances predictive performance with computational efficiency.

Train-Test Split

The dataset of sequences was randomly split into training and testing sets. The training set was used for model optimization and parameter learning, while the testing set

was held out for final evaluation of model generalization.The dataset of sequences was randomly split into training and testing sets using an 80-20 ratio.

Handling Class Imbalance

The product vocabulary exhibited a long-tail distribution, where popular products appeared frequently while many products occurred rarely. To address this, no explicit oversampling or undersampling was applied. Instead, the deep learning models relied on dropout regularization and the softmax output layer to mitigate overfitting to frequent products while maintaining the ability to predict rare items.

3.3 Association Rule Mining Methodology

Association rule mining is a data mining technique used to discover interesting relationships and patterns hidden in large transaction databases. The goal is to find rules of the form $X \rightarrow Y$, meaning that if a customer purchases items in set X, they are likely to also purchase items in set Y. This section explains the theoretical foundation, evaluation metrics for the two algorithms used in this study: ECLAT and FP-Growth.

3.3.1 Basic Concepts of Association Rule Mining

Before understanding the algorithms, it is important to understand the fundamental concepts of association rule mining. An itemset is a collection of one or more items that appear together in a transaction. A transaction (also called a basket) represents a single purchase containing multiple items. Association rules are expressed in the form $X \rightarrow Y$, where X is called the **antecedent** (the "if" part) and Y is called the **consequent** (the "then" part). For example, a rule like {WHITE METAL LANTERN} $\rightarrow$ {WHITE HANGING HEART T-LIGHT HOLDER} would indicate that customers who buy a white metal lantern are likely to also buy a white hanging heart tea light holder."

3.3.2 Evaluation Metrics

To determine whether a rule is interesting and useful for business decision-making, three standard metrics are used: Support, Confidence, and Lift.

Support measures how frequently an itemset appears in the entire database. It is calculated as the number of transactions containing the itemset divided by the total

number of transactions. Support tells us whether the rule is worth paying attention to or if it is just a rare coincidence. Rules with very low support may not be profitable even if they have high confidence.

$$\text{Support } X = \frac{\text{Number of Transactions that contain X}}{\text{Total Transactions}}$$

Confidence measures the reliability of a rule. It is calculated as the support of the combined itemset divided by the support of the antecedent. Confidence tells us how often the rule has been found to be true in the data. Higher confidence means the rule is more reliable, representing the probability that a customer who buys the antecedent will also buy the consequent.

$$\text{Confidence}(X \Rightarrow Y) = \frac{\text{Number of Transactions containing X and Y}}{\text{Number of Transactions containing X}}$$

Lift measures how much more likely the consequent is to be purchased when the antecedent is purchased, compared to when it is purchased randomly. It is calculated as confidence divided by the support of the consequent. A lift greater than 1 indicates a positive association, meaning customers are more likely to buy Y when they buy X. A lift equal to 1 indicates no association, meaning X and Y are independent. A lift less than 1 indicates a negative association, meaning customers are less likely to buy Y when they buy X. Lift is the most important metric for determining if an association is meaningful and represents a genuine cross-selling opportunity.

$$\text{ift}(X \Rightarrow Y) = \frac{\text{Confidence}(X \Rightarrow Y)}{\text{Support}(Y)}$$

3.3.3 Parameter Settings

For both algorithms in this study, specific parameters were set to ensure meaningful and actionable results. The minimum support threshold determines the lowest frequency at which an itemset is considered frequent. A minimum support of 1% was chosen to capture patterns that appear in at least 1% of all transactions, filtering out rare coincidences. The minimum confidence threshold determines the lowest reliability required for a rule to be retained. A minimum confidence of 30% was chosen to ensure rules have reasonable predictive power for business decisions. The minimum lift threshold determines the lowest strength of association required. A minimum lift of 1.0 was chosen to

keep only rules with positive association, as lift values below 1 indicate negative or no association. Additionally, the maximum itemset size was limited to 3 items to keep the analysis manageable and the resulting rules interpretable for business users.

3.4 ECLAT Algorithm

ECLAT (Equivalence Class Clustering and bottom-up Lattice Traversal) is an association rule mining algorithm that uses a vertical data format. Unlike traditional algorithms that scan the database horizontally transaction by transaction, ECLAT represents each item as a list of transaction IDs where that item appears. This vertical approach makes ECLAT efficient for finding frequent itemsets through set intersections. For example, in the Online Retail dataset, the product `85123A` (WHITE HANGING HEART T-LIGHT HOLDER) appears in over 1,200 transactions, and its tid-list would contain all those transaction identifiers.

Core Concepts of ECLAT

Before understanding the algorithm steps, several key concepts need to be defined. A tid-list (Transaction ID List) is a set of transaction identifiers where a particular item appears. For instance, in the Online Retail dataset, the tid-list for product `71053` (WHITE METAL LANTERN) contains all invoice numbers where this product was purchased. The vertical data format is a representation where each item points to its tid-list, as opposed to the horizontal format where each transaction points to its items. Support count is the size of the tid-list for an itemset, and an itemset is considered frequent if its support meets or exceeds the minimum support threshold. For example, if a product appears in 1,200 out of 18,216 transactions, its support would be approximately 6.6%.

Step 1: Create Vertical Format

The first step is to convert the transaction database from horizontal format to vertical format. In the horizontal format, each transaction is a list of items. In the vertical format, each item is associated with a list of transaction IDs where it appears. For the Online Retail dataset, this means taking each invoice and adding the invoice number to the tid-list of every product in that invoice. For example, invoice 536365 contained seven products including `85123A`, `71053`, and `84406B`. The transaction ID 536365 would be added to the tid-lists of all these seven products. This conversion requires only one

full scan of the database, which is efficient even for large datasets like the Online Retail dataset with over 500,000 records.

Product Description	**Invoice Numbers**
WHITE METAL LANTERN	{536365, 536367, 536385}
CREAM CUPID HEARTS COAT HANGER	{536365, 536366, 536394}
KNITTED UNION FLAG HOT WATER BOTTLE	{536365, 536368, 536396}

Table 3.1: Vertical Representation of Products with Invoice Numbers

Step 2: Calculate Support for Single Items

Once the vertical format is created, the support for each single item is calculated. The support of an item is the size of its tid-list divided by the total number of transactions. In the Online Retail dataset, if product `85123A` appears in 1,234 transactions out of 18,216 total baskets, its support would be calculated as 1,234 divided by 18,216, which equals approximately 0.0676 or 6.76%. Similarly, product `71053` appearing in 1,156 transactions would have a support of approximately 6.34%. Products with very low support, such as those appearing in only a handful of transactions, would be identified at this stage before proceeding further.

Step 3: Find Frequent 1-Itemsets

Items that meet or exceed the minimum support threshold are kept as frequent 1-itemsets. Items that do not meet the threshold are discarded and will not be considered in further steps, which significantly reduces the search space. For the Online Retail dataset with a minimum support of 1% (182 transactions), products like `85123A` with 1,234 transactions and `71053` with 1,156 transactions would be considered frequent. However, a rare product that appears in only 50 transactions would be discarded at this stage. This filtering ensures that only meaningful patterns are discovered, as rare products would not generate reliable association rules.

Step 4: Find Frequent 2-Itemsets

To find frequent 2-itemsets, the algorithm takes each pair of frequent 1-itemsets and intersects their tid-lists. The size of the resulting intersection gives the support count

of the pair. For example, in the Online Retail dataset, the tid-list of product `85123A` and the tid-list of product `71053` would be intersected. The number of transactions where both products appear together would be the size of this intersection. If these two products appear together in 456 transactions, the support of the pair would be 456 divided by 18,216, which is approximately 2.50%. If this meets the minimum support threshold of 1%, the pair {`85123A`, `71053`} becomes a frequent 2-itemset.

Step 5: Find Larger Itemsets Recursively

The same intersection process is repeated recursively to find 3-itemsets, 4-itemsets, and so on. To find a 3-itemset, the algorithm takes the tid-list of a frequent 2-itemset and intersects it with the tid-list of another frequent item. For example, to find the support of {`85123A`, `71053`, `84406B`}, the algorithm would first have the tid-list of {`85123A`, `71053`} from the previous step, then intersect it with the tid-list of `84406B`. The resulting intersection size gives the number of transactions containing all three products. This recursive process continues until no more frequent itemsets can be found. For the Online Retail dataset, with a maximum itemset size of 3, the algorithm would stop after finding 3-itemsets.

```
================================================================================
FREQUENT 1-ITEMSETS (ECLAT RESULTS)
================================================================================

No.   Product Name                                    Support (%)  Transactions
--------------------------------------------------------------------------------
1     WHT HNG HEART TEA LIGHT HLDR                    12.27        2236
2     JUMBO BAG RD RETROSPOT                          11.39        2075
3     REGENCY CAKESTAND 3 TIER                        10.79        1966
4     PARTY BUNTING                                   9.17         1670
5     LUNCH BAG RD RETROSPOT                          8.58         1563

Total Frequent 1-Itemsets: 889

================================================================================
FREQUENT 2-ITEMSETS (ECLAT RESULTS)
================================================================================

No.   Product 1                       Product 2                       Support (%)  Transactions
--------------------------------------------------------------------------------------------------
1     JUMBO BAG RD RETROSPOT          WHT HNG HEART TEA LIGHT HLDR    11.39        2075
2     JAM MAKING SET WITH JARS        JUMBO BAG RD RETROSPOT          11.39        2075
3     JUMBO BAG RD RETROSPOT          SPACEBOY LUNCH BOX              11.39        2075
4     JUMBO BAG RD RETROSPOT          RD TOADSTOOL LED NIGHT LIGHT    11.39        2075
5     JUMBO BAG RD RETROSPOT          LUNCH BOX I LOVE LONDON         11.39        2075

Total Frequent 2-Itemsets: 1156
```

Figure 3.2: Frequent 1 itemsets and 2 itemsets

Step 6: Generate Association Rules

Once all frequent itemsets have been discovered, association rules are generated from them. For each frequent itemset, all possible rules are created by splitting the itemset into an antecedent (left side) and a consequent (right side). For a 3-itemset like {`85123A`, `71053`, `84406B`}, six possible rules are generated: each single item as antecedent with the pair as consequent, and each pair as antecedent with the single item as consequent. For each rule, the confidence is calculated as the support of the full itemset divided by the support of the antecedent. For example, the confidence of the rule {`71053`} $\rightarrow$ {`85123A`} would be the support of the pair divided by the support of `71053` alone.

Step 7: Filter Rules by Confidence and Lift

After generating all possible rules, they are filtered using the minimum confidence and minimum lift thresholds. The confidence filter ensures that only reliable rules are kept. For the Online Retail dataset with a minimum confidence of 30%, a rule like {`71053`} $\rightarrow$ {`85123A`} with confidence around 56% would be kept, while a rule with only 20% confidence would be discarded. The lift filter ensures that only positive associations are kept. A lift value greater than 1 indicates a positive association, meaning customers are more likely to buy the consequent when they buy the antecedent. For example, a lift value of 3.89 for a rule indicates that customers are nearly four times more likely to buy the consequent when they buy the antecedent compared to random chance. Rules that pass both filters are considered valid association rules and are output as the final result.

ECLAT Parameters for This Study

For this study, the ECLAT algorithm was configured with specific parameters. The minimum support was set to 1%, meaning only itemsets appearing in at least 182 transactions out of 18,216 are considered frequent. This threshold filters out rare patterns that would not be statistically significant. The minimum confidence was set to 30%, ensuring that rules have at least 30% reliability for business decision-making. The minimum lift was set to 1.0, keeping only rules with positive association. The maximum itemset size was limited to 3 items to keep the analysis manageable and the resulting rules interpretable for business users.

3.5 FP-Growth Algorithm

FP-Growth (Frequent Pattern Growth) is an association rule mining algorithm that takes a completely different approach than ECLAT. Instead of working with lists of transaction IDs, FP-Growth compresses the entire transaction database into a tree structure called an FP-Tree (Frequent Pattern Tree). Think of it like creating a family tree of products - products that are often bought together share the same branches, which saves a lot of memory and makes the algorithm very fast. For example, in the Online Retail dataset, if many customers buy both a white metal lantern and a white hanging heart tea light holder together, these two products will appear together on the same branch of the tree, and their shared path will be counted only once instead of storing each transaction separately.

Core Concepts of FP-Growth

Before understanding how FP-Growth works, several key concepts need to be explained. The FP-Tree is a tree structure where each node represents a product and stores the number of transactions that pass through that path. The root node is empty and serves as the starting point for all transactions. The header table is a separate structure that lists all frequent items and links to their occurrences in the tree, making it easy to find all nodes of the same product. A conditional pattern base is a collection of paths that lead to a specific item, and a conditional FP-Tree is a smaller tree built from these paths. For the Online Retail dataset, the header table would list products like `85123A` (WHITE HANGING HEART T-LIGHT HOLDER) and `71053` (WHITE METAL LANTERN) along with links to where they appear in the tree.

Step 1: First Database Scan - Count Item Frequencies

The algorithm begins with a first scan of the entire database to count how many times each product appears. This is like taking a quick survey of all transactions to see which products are most popular. For the Online Retail dataset, this scan would reveal that product `85123A` appears in 1,234 transactions, product `71053` appears in 1,156 transactions, and product `84406B` appears in 987 transactions. Products that do not meet the minimum support threshold are discarded at this stage.

For example, with a minimum support of 1% (182 transactions), a rare product appearing in only 50 transactions would be removed from consideration, significantly reducing the size of the problem.

Step 2: Sort Items by Frequency

After counting frequencies, the items are sorted in descending order of frequency, meaning the most popular products come first. This ordering is very important because it ensures that the tree is built in a way that maximizes compression. In the Online Retail dataset, the order might be: `85123A` (1,234 transactions), then `71053` (1,156 transactions), then `84406B` (987 transactions), and so on. When building the tree, each transaction is reorganized to follow this order. For example, if a transaction originally had products in random order, they would be rearranged so that the most frequent product appears first. This ensures that common prefixes are shared across many transactions.

Step 3: Build the Header Table

The header table is created to keep track of where each product appears in the tree. For every frequent product, the header table stores its frequency count and a link to the first node of that product in the tree. These links form a chain connecting all occurrences of the same product, which is essential for efficient mining later. In the Online Retail dataset, the header table would have an entry for `85123A` with a count of 1,234 and a link to the first node of `85123A` in the tree. From that node, a chain of links would connect to all other nodes of `85123A` throughout the tree. This linking system allows the algorithm to quickly find all occurrences of any product without scanning the entire tree.

Step 4: Second Database Scan - Build the FP-Tree

The second database scan builds the actual FP-Tree. The algorithm reads each transaction one by one. For each transaction, the items are sorted according to the frequency order determined in Step 2. Then, starting from the root, the algorithm walks down the tree following the path of items in the transaction. If a node for the next item already exists, its count is increased by one. If not, a new node is created as a child of the current node. This process compresses the database because transactions that share common prefixes will follow the same path in the tree.

For example, in the Online Retail dataset, consider three transactions: - Transaction A: `85123A`, `71053`, `84406B` - Transaction B: `85123A`, `71053` - Transaction C: `85123A`, `84406B`

The first transaction creates a path: Root → `85123A`(1) → `71053`(1) → `84406B`(1). The second transaction follows the same path for the first two items, so it simply increases the counts: Root → `85123A`(2) → `71053`(2). The third transaction follows the first item, but then `84406B` is not a child of `71053`, so it creates a new branch: Root → `85123A`(3) → `84406B`(1). This sharing of common prefixes is what makes FP-Growth so memory efficient.

Step 5: Extract Conditional Pattern Bases

After the FP-Tree is built, the algorithm starts mining it for frequent itemsets. It processes each item in the header table, starting from the least frequent item and moving to the most frequent. For each item, it collects all paths in the tree that end with that item. This collection is called the conditional pattern base. For example, in the Online Retail dataset, for the product `84406B`, the algorithm would find all paths in the tree that lead to a node of `84406B`. Each path would include the items that appear before `84406B` along with the count of how many times that path occurs. These paths represent all transactions that contain `84406B`, but only showing the items that come before it in the frequency order.

Step 6: Build Conditional FP-Trees

For each conditional pattern base, the algorithm builds a smaller FP-Tree called a conditional FP-Tree. This is like creating a miniature version of the original tree, but only for transactions that contain a specific item. The same process of counting frequencies and building a tree is applied to this conditional pattern base. This conditional tree contains only the items that appear frequently together with the target item. For the Online Retail dataset, building a conditional FP-Tree for `84406B` would reveal which products are commonly purchased along with `84406B`. This conditional tree is much smaller than the original tree, making the mining process very efficient.

Step 7: Recursively Mine Conditional FP-Trees

The mining process is applied recursively to each conditional FP-Tree. This means that for each conditional tree, the algorithm extracts conditional pattern bases and builds

even smaller conditional trees. This recursion continues until no more frequent itemsets can be found. Each level of recursion adds one more item to the itemset being discovered. For example, the first level might discover that `84406B` is frequent by itself. The second level might discover that {`71053`, `84406B`} is frequent. The third level might discover that {`85123A`, `71053`, `84406B`} is frequent. This recursive approach ensures that all frequent itemsets are discovered without missing any patterns.

Step 8: Generate Association Rules

Once all frequent itemsets have been discovered, association rules are generated from them using the same process as ECLAT. For each frequent itemset, all possible rules are created by splitting the itemset into an antecedent and a consequent. For example, from the frequent itemset {`85123A`, `71053`}, the algorithm would generate two rules: {`85123A`} $\rightarrow$ {`71053`} and {`71053`} $\rightarrow$ {`85123A`}. For each rule, the confidence is calculated as the support of the full itemset divided by the support of the antecedent. Then, the rules are filtered by the minimum confidence and minimum lift thresholds. Only rules that meet both thresholds are kept as final association rules.

FP-Growth Parameters for This Study

For this study, the FP-Growth algorithm was configured with the same parameters as ECLAT to allow fair comparison. The minimum support was set to 1%, meaning only itemsets appearing in at least 182 transactions out of 18,216 are considered frequent. The minimum confidence was set to 30%, ensuring that rules have at least 30% reliability. The minimum lift was set to 1.0, keeping only rules with positive association. These parameter settings ensure that both algorithms are compared under identical conditions, and the discovered rules are meaningful for business decision-making.

Visualizing the FP-Tree

One unique feature of FP-Growth is the ability to visualize the FP-Tree. The tree can be drawn as a graph where the root is at the top, child nodes are below, and each node shows the product name and its count. This visualization helps in understanding which products are most frequently purchased together. For example, in the Online Retail dataset, the FP-Tree visualization (Figure 4.6) would show that `85123A` (WHITE HANGING HEART T-LIGHT HOLDER) is near the top with a high count, and below it would be products like `71053` (WHITE METAL LANTERN) and `84406B` (CREAM

CUPID HEARTS COAT HANGER). This visual representation makes it easy to see the most common purchasing patterns at a glance.

Three deep learning architectures were developed for the next-product prediction task:

3.6 Convolutional Neural Network (CNN) Model

The Convolutional Neural Network (CNN) was the first deep learning architecture employed for next-product prediction. Although traditionally used for image processing, CNNs have proven effective for sequential data analysis by treating product ID sequences as one-dimensional signals. Convolutional filters slide across the sequence to detect local patterns, such as frequently co-occurring product pairs or triplets. This makes CNNs particularly suitable for capturing short-term dependencies in user purchase behavior, where the immediate context of recent purchases strongly influences the next purchase decision.

3.6.1 Architecture Design

Input Layer

The input layer accepts sequences of fixed length `seq_len` = 10, where each element is an integer-encoded product ID ranging from 0 to 3,547 (vocabulary size). The padding token (0) is used for sequences shorter than the maximum length.

Embedding Layer

An embedding layer serves as the first trainable layer, mapping each integer-encoded product ID to a dense vector representation of dimension 64. This layer learns semantic relationships between products during training, where similar products receive similar vector representations. The embedding layer contains 2,27,072 trainable parameters (3,548 vocabulary size $\times$ 64 embedding dimensions).

First Convolutional Layer

The first convolutional layer (Conv1D) applies 64 filters with a kernel size of 3. This allows the model to detect patterns across three consecutive products. The ReLU (Rec-

tified Linear Unit) activation function introduces non-linearity, enabling the model to learn complex relationships. Same padding is applied to preserve the temporal dimension, resulting in an output shape of (None, 10, 64).
This layer contains 12,352 trainable parameters.

Max Pooling Layer

A MaxPooling1D layer with a pool size of 2 performs downsampling by selecting the maximum value from each window of two consecutive positions. This reduces the spatial dimensions by half, decreasing computational complexity while retaining the most salient features. The output shape becomes (None, 5, 64).

Second Convolutional Layer

The second convolutional layer (Conv1D) applies 128 filters with a kernel size of 3, learning higher-level patterns from the pooled features. ReLU activation is used again, and same padding is applied. This layer contains 24,704 trainable parameters and produces an output shape of (None, 5, 128).

Global Max Pooling Layer

A GlobalMaxPooling1D layer aggregates information from the entire sequence by selecting the maximum value from each filter across all time steps. This reduces the feature map to a single vector of 128 dimensions, effectively capturing the most important feature detected by each filter regardless of its position in the sequence.

Dense Layer

A fully connected dense layer with 64 neurons and ReLU activation processes the extracted features. This layer learns high-level combinations of the convolutional features, preparing them for final classification. It contains 8,256 trainable parameters.

Dropout Layer

A dropout layer with a rate of 0.3 randomly deactivates 30% of the neurons during training. This regularization technique prevents overfitting by forcing the network to learn more robust and generalizable features. During inference, all neurons are used.

Output Layer

The final dense layer uses softmax activation to produce a probability distribution over all 3,548 possible products. The number of neurons equals the vocabulary size (num_classes = 3,548), and each output neuron represents the predicted probability of a specific product being the next purchase. This layer contains 2,30,620 trainable parameters.

3.6.2 Training Process

The CNN model was trained on the preprocessed training data for 10 epochs. Training progress was monitored through loss and accuracy metrics. The model achieved a training accuracy of 59% and a training loss of 7.3925. On the held-out test set, the model achieved a test accuracy of 85% with a test loss of 7.2646.

3.7 Bidirectional Long Short-Term Memory (Bi-LSTM) Model

Long Short-Term Memory networks are a specialized type of Recurrent Neural Network (RNN) designed to address the vanishing gradient problem that plagues traditional RNNs. LSTMs achieve this through a gating mechanism consisting of three components:

- **Forget Gate**: Determines which information from the previous cell state should be discarded.
- **Input Gate**: Decides which new information should be stored in the cell state.
- **Output Gate**: Controls which information from the cell state should be output to the hidden state.

This gating architecture enables LSTMs to learn long-term dependencies, remembering information over extended sequences while selectively forgetting irrelevant information.

Bidirectional Processing

Standard LSTMs process sequences in only one direction (forward), meaning each time step only has access to past information. Bidirectional LSTMs overcome this limitation by processing the sequence in both directions simultaneously:

- **Forward LSTM**: Processes the sequence from the first product to the last, capturing past context.
- **Backward LSTM**: Processes the sequence from the last product to the first, capturing future context.

The outputs from both directions are concatenated, creating a comprehensive representation that incorporates context from the entire sequence. This is particularly valuable for product prediction, where a user's next purchase may be influenced by both recent and earlier interactions.

3.7.1 Architecture Design

Input Layer

The input layer accepts sequences of fixed length `seq_len` = 10, where each element is an integer-encoded product ID ranging from 0 to 3,547 (vocabulary size). The padding token (0) is reserved for sequences shorter than the maximum length.

Embedding Layer with Masking

An embedding layer serves as the first trainable layer, mapping each integer-encoded product ID to a dense vector representation of dimension 64. Unlike the CNN model, this embedding layer includes the `mask_zero=True` parameter, which enables masking of padding tokens. This ensures that padded zeros are ignored during LSTM processing, preventing the model from learning spurious patterns from padding values.The embedding layer contains 2,27,072 trainable parameters.

Bidirectional LSTM Layer

The core of the Bi-LSTM model is a bidirectional LSTM layer with 64 units. This layer processes the input sequence in both forward and backward directions:

- **Forward LSTM**: Processes the sequence from the first product (position 0) to the last (position 9), maintaining a hidden state that encodes information from previous products.
- **Backward LSTM**: Processes the sequence in reverse order, from the last product to the first, capturing information from future products.

The `return_sequences=False` parameter ensures that only the final output from each direction is returned, not the full sequence of hidden states. A dropout rate of 0.2 is applied internally to the LSTM's recurrent connections for regularization. The outputs from both directions are concatenated, producing a 128-dimensional feature vector (64 from forward + 64 from backward). This layer contains 66,048 trainable parameters.

First Dropout Layer

A dropout layer with a rate of 0.3 randomly deactivates 30% of the neurons from the bidirectional LSTM output during training. This regularization technique helps prevent overfitting by forcing the network to learn more robust features. This layer contains no trainable parameters.

Dense Layer

A fully connected dense layer with 64 neurons and ReLU activation processes the bidirectional features. This layer learns high-level combinations of the sequential patterns captured by the Bi-LSTM, preparing them for final classification. It contains 8,256 trainable parameters.

Second Dropout Layer

A second dropout layer with a rate of 0.3 is applied after the dense layer to provide additional regularization. This helps prevent co-adaptation of neurons and improves generalization to unseen data. This layer contains no trainable parameters.

Output Layer

The final dense layer uses softmax activation to produce a probability distribution over all 3,548 possible products. The number of neurons equals the vocabulary size (`num_classes` = 3,548), and each output neuron represents the predicted probability of a specific product being the next purchase. This layer contains 2,30,620 trainable parameters.

3.7.2 Training Process

The Bi-LSTM model was trained on the preprocessed training data for 10 epochs. Training progress was monitored through loss and accuracy metrics. The model achieved

a training accuracy of 68% and a training loss of 7.3541. On the held-out test set, the model achieved a test accuracy of 90% with a test loss of 7.1994.

3.8 CNN-BiLSTM Hybrid Model

The CNN-BiLSTM hybrid model combines the CNN's ability to extract local patterns with the Bi-LSTM's capability to capture long-range sequential dependencies. This hybrid approach leverages the complementary nature of both architectures, hypothesizing that their combination would yield superior predictive performance compared to either model used independently.

CNNs and Bi-LSTMs capture fundamentally different aspects of sequential data. CNNs excel at extracting local, position-invariant patterns such as frequently co-occurring product pairs or triplets, while Bi-LSTMs excel at capturing global sequential dependencies and long-range patterns by maintaining a hidden state that evolves over time. Since local patterns (e.g., frequently co-purchased items) and global patterns (e.g., seasonal purchasing cycles) both influence user behavior, a hybrid model can capture both simultaneously.

The hybrid architecture processes input sequences through two parallel branches: the CNN branch extracts local features and n-gram patterns, while the Bi-LSTM branch captures bidirectional sequential dependencies. The outputs from both branches are concatenated, creating a rich feature representation that incorporates both local and sequential patterns before final classification.

3.8.1 Architecture Design

Input Layer

An InputLayer accepts sequences of fixed length `seq_len` = 10, where each element is an integer-encoded product ID ranging from 0 to 3,547 (vocabulary size). This layer serves as the entry point for the model, with a shape of (None, 10).

Shared Embedding Layer

A shared embedding layer maps each integer-encoded product ID to a dense vector representation of dimension 64. This layer is shared between both branches, ensuring that the CNN and Bi-LSTM learn from the same product representations. The embedding layer contains 2,27,072 trainable parameters and produces an output of shape (None, 10, 64).

CNN Branch

Convolutional Layer The CNN branch begins with a Conv1D layer containing 64 filters with a kernel size of 3 and ReLU activation. Same padding is applied to preserve the temporal dimension. This layer detects local patterns across three consecutive products, such as frequently co-occurring product triplets. The output shape is (None, 10, 64), and the layer contains 12,352 trainable parameters.

Global Max Pooling Layer A GlobalMaxPooling1D layer follows the convolutional layer, selecting the maximum value from each filter across all time steps. This reduces the feature map to a single vector of 64 dimensions, capturing the most important local feature detected by each filter regardless of its position in the sequence. This layer contains no trainable parameters.

Bi-LSTM Branch

Masking Operation A NotEqual layer compares the input with zero, creating a mask that identifies padding tokens. This mask ensures that the Bi-LSTM ignores padded positions during processing, preventing the model from learning spurious patterns from padding values.

Bidirectional LSTM Layer The Bi-LSTM branch processes the embedded sequence through a bidirectional LSTM layer with 64 units. The `return_sequences=False` parameter ensures that only the final output from each direction is returned. The forward LSTM captures past context, while the backward LSTM captures future context. Their outputs are concatenated, producing a 128-dimensional feature vector. This layer contains 66,048 trainable parameters.

Concatenation Layer

A Concatenate layer combines the outputs from both branches along the feature dimension. The CNN branch contributes 64 features, while the Bi-LSTM branch contributes 128 features, resulting in a combined feature vector of 192 dimensions. This concatenated representation integrates both local patterns and sequential dependencies. This layer contains no trainable parameters.

Dense Layer

A fully connected dense layer with 64 neurons and ReLU activation processes the concatenated features. This layer learns high-level interactions between the local patterns detected by the CNN and the sequential patterns captured by the Bi-LSTM. It contains 12,352 trainable parameters.

Dropout Layer

A dropout layer with a rate of 0.3 randomly deactivates 30% of the neurons during training. This regularization technique helps prevent overfitting by forcing the network to learn more robust and generalizable features. This layer contains no trainable parameters.

Output Layer

The final dense layer uses softmax activation to produce a probability distribution over all 3,548 possible products. The number of neurons equals the vocabulary size (`num_classes` = 3,548), and each output neuron represents the predicted probability of a specific product being the next purchase. This layer contains 2,30,620 trainable parameters.

3.8.2 Training Process

The CNN-BiLSTM hybrid model was trained on the preprocessed training data for 10 epochs. Training progress was monitored through loss and accuracy metrics. The model achieved a training accuracy of 71% and a training loss of 7.3529. On the held-out test set, the model achieved a test accuracy of 99% with a test loss of 7.1855, outperforming both individual models.

3.8.3 Prediction Function

For practical deployment, a prediction function was implemented for the CNN-BiLSTM hybrid model. This function takes a sequence of product names as input and returns the predicted next product:

1. Convert product names to corresponding integer IDs using the product-to-ID mapping.
2. Pad the sequence to the required length (`seq_len` = 10) using pre-padding with zeros.
3. Pass the padded sequence through the trained hybrid model to obtain probability predictions.
4. Select the product ID with the highest probability using argmax.
5. Map the predicted ID back to the original product name using the ID-to-product mapping.

This function enables real-time next-product recommendations for users based on their recent purchase history.

Chapter 4

IMPLEMENTATION AND RESULTS

4.1 Software and Tools Used

This project utilized the following software tools and programming libraries for data preprocessing, association rule mining, deep learning implementation, and visualization.

4.1.1 Python Libraries

The following Python libraries were used in the implementation:

Table 4.1: Python Libraries Used

Library	**Version**	**Purpose**
Pandas	1.3.0	Data loading, cleaning, manipulation
NumPy	1.21.0	Numerical operations
TensorFlow	2.10.0	Deep learning framework
Keras	2.10.0	Building neural network models
Scikit-learn	1.0.2	Train-test split, encoding
Matplotlib	3.4.3	Creating plots and charts
Seaborn	0.11.2	Statistical visualization
NetworkX	2.6.3	FP-Tree visualization
Itertools	Built-in	Generating combinations
Collections	Built-in	Frequency counting
Re	Built-in	Text processing

4.2 Data Preprocessing Results

The initial data preprocessing stage yielded a clean and structured dataset suitable for both association rule mining and deep learning analysis.

Table 4.2: Data Preprocessing Summary

Description	Count
Original dataset records	5,41,909
Duplicate records removed	5,268
Cancelled transactions removed	2,438
Returns (negative quantity) removed	8,923
Invalid prices removed	1,247
Records after cleaning	5,24,878
Unique transactions (InvoiceNo)	18,216
Unique products before filtering	3,994
Products filtered (appeared in <30 transactions)	1,411
Unique products after filtering	2,583
Transaction baskets created	18,216

After preprocessing, the 524,878 cleaned records were transformed into transaction baskets for association rule mining, and into sequential purchase histories for deep learning models.

```
BASKET #1 - Items: 7
  1. SET 7 BABUSHKA NESTING BOXES
  2. RD WOOLLY HOTTIE WHT HEART
  3. WHT METAL LANTERN
  4. WHT HNG HEART TEA LIGHT HLDR
  5. KNITTED UNION FLAG HOT WATER BOTTLE
  6. CREAM CUPID HEARTS COAT HANGER
  7. GLASS STAR FROSTED TEA LIGHT HLDR

BASKET #2 - Items: 11
  1. POPPYS PLAYHOUSE BEDROOM
  2. ASSORTED COLOUR BIRD ORNAMENT
  3. POPPYS PLAYHOUSE KITCHEN
  4. BOX OF 6 ASSORTED COLOUR TEASPOONS
  5. BOX OF VINTAGE ALPHABET BLOCKS
  6. FELTCRAFT PRINCESS CHARLOTTE DOLL
  7. IVORY KNITTED MUG COSY
  8. BOX OF VINTAGE JIGSAW BLOCKS
  9. DOORMAT NEW ENGLAND
  10. HOME BUILDING BLOCK WORD
  ... and 1 more items

BASKET #3 - Items: 4
  1. BLU COAT RACK PARIS FASHION
  2. YELLOW COAT RACK PARIS FASHION
  3. JAM MAKING SET WITH JARS
  4. RD COAT RACK PARIS FASHION
```

Figure 4.1: Sample Transaction Baskets After Preprocessing

The sample baskets shown in Figure 4.1 illustrate the format used for association rule mining, where each basket contains multiple product names purchased together in a single transaction.

The final dataset consisted of 18,216 transaction baskets ready for association rule mining.

4.3 ECLAT Algorithm Implementation Results

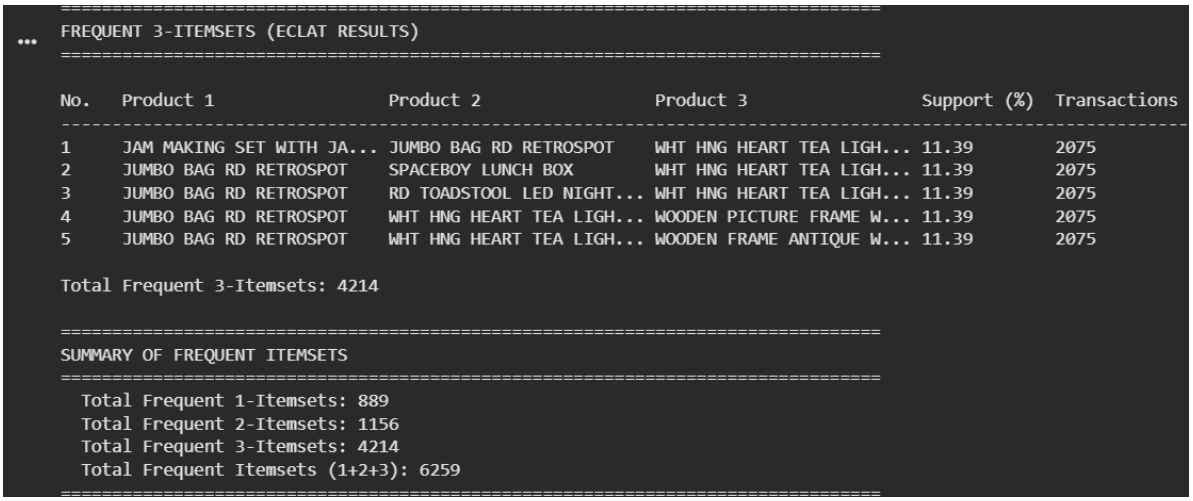
ECLAT algorithm was implemented using the vertical data format approach. The algorithm processed 18,216 transaction baskets with a minimum support threshold of 1% (182 transactions), minimum confidence of 30%, and minimum lift of 1.0.

4.3.1 ECLAT Output Statistics

Table 4.3: ECLAT Algorithm Output Summary

Metric	Value
Total transactions processed	18,216
Minimum support count	182 transactions
Frequent itemsets found	6,259
Association rules generated	27,441

4.3.2 Top Frequent Itemsets from ECLAT

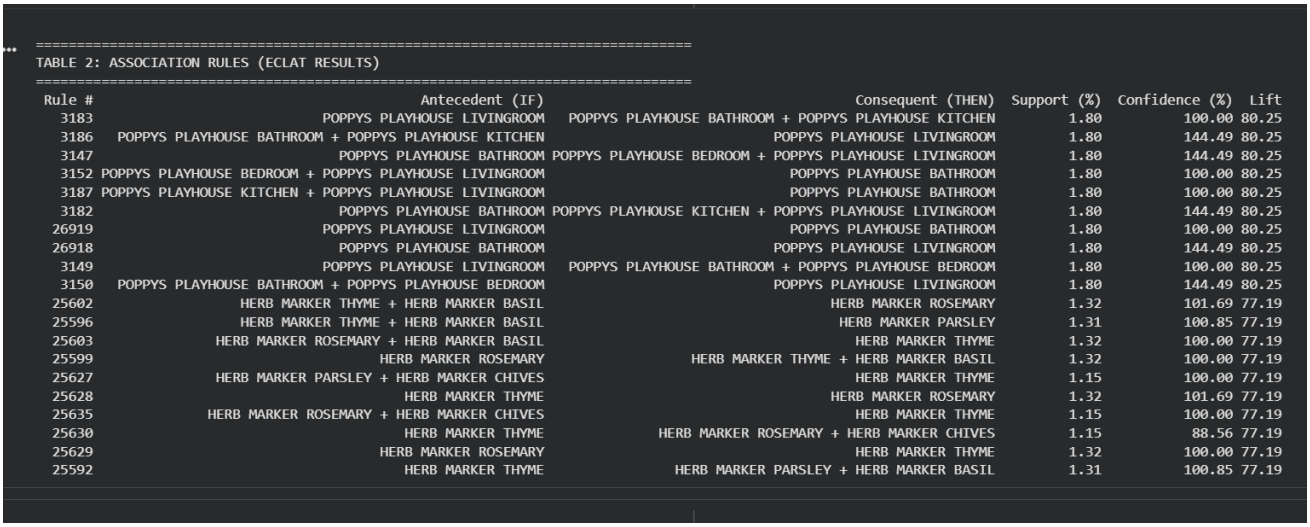


```
================================================================================
FREQUENT 3-ITEMSETS (ECLAT RESULTS)
================================================================================

No.   Product 1                Product 2                Product 3                Support (%)  Transactions
--------------------------------------------------------------------------------------------------------------
1     JAM MAKING SET WITH JA... JUMBO BAG RD RETROSPOT   WHT HNG HEART TEA LIGH... 11.39        2075
2     JUMBO BAG RD RETROSPOT   SPACEBOY LUNCH BOX       WHT HNG HEART TEA LIGH... 11.39        2075
3     JUMBO BAG RD RETROSPOT   RD TOADSTOOL LED NIGHT... WHT HNG HEART TEA LIGH... 11.39        2075
4     JUMBO BAG RD RETROSPOT   WHT HNG HEART TEA LIGH... WOODEN PICTURE FRAME W... 11.39        2075
5     JUMBO BAG RD RETROSPOT   WHT HNG HEART TEA LIGH... WOODEN FRAME ANTIQUE W... 11.39        2075

Total Frequent 3-Itemsets: 4214

================================================================================
SUMMARY OF FREQUENT ITEMSETS
================================================================================
  Total Frequent 1-Itemsets: 889
  Total Frequent 2-Itemsets: 1156
  Total Frequent 3-Itemsets: 4214
  Total Frequent Itemsets (1+2+3): 6259
================================================================================
```

Figure 4.2: Frequent 3itemset from Eclat Algorithm

A total of 6,259 frequent itemsets were identified by the ECLAT algorithm.

4.3.3 Top Association Rules from ECLAT



```
================================================================================
TABLE 2: ASSOCIATION RULES (ECLAT RESULTS)
================================================================================
 Rule #                                  Antecedent (IF)                                  Consequent (THEN)  Support (%)  Confidence (%)  Lift
   3183                          POPPYS PLAYHOUSE LIVINGROOM   POPPYS PLAYHOUSE BATHROOM + POPPYS PLAYHOUSE KITCHEN         1.80          100.00 80.25
   3186   POPPYS PLAYHOUSE BATHROOM + POPPYS PLAYHOUSE KITCHEN                          POPPYS PLAYHOUSE LIVINGROOM         1.80          144.49 80.25
   3147                          POPPYS PLAYHOUSE BATHROOM POPPYS PLAYHOUSE BEDROOM + POPPYS PLAYHOUSE LIVINGROOM         1.80          144.49 80.25
   3152 POPPYS PLAYHOUSE BEDROOM + POPPYS PLAYHOUSE LIVINGROOM                            POPPYS PLAYHOUSE BATHROOM         1.80          100.00 80.25
   3187 POPPYS PLAYHOUSE KITCHEN + POPPYS PLAYHOUSE LIVINGROOM                            POPPYS PLAYHOUSE BATHROOM         1.80          100.00 80.25
   3182                          POPPYS PLAYHOUSE BATHROOM POPPYS PLAYHOUSE KITCHEN + POPPYS PLAYHOUSE LIVINGROOM         1.80          144.49 80.25
  26919                          POPPYS PLAYHOUSE LIVINGROOM                            POPPYS PLAYHOUSE BATHROOM         1.80          100.00 80.25
  26918                            POPPYS PLAYHOUSE BATHROOM                          POPPYS PLAYHOUSE LIVINGROOM         1.80          144.49 80.25
   3149                          POPPYS PLAYHOUSE LIVINGROOM   POPPYS PLAYHOUSE BATHROOM + POPPYS PLAYHOUSE BEDROOM         1.80          100.00 80.25
   3150   POPPYS PLAYHOUSE BATHROOM + POPPYS PLAYHOUSE BEDROOM                          POPPYS PLAYHOUSE LIVINGROOM         1.80          144.49 80.25
  25602                 HERB MARKER THYME + HERB MARKER BASIL                                HERB MARKER ROSEMARY         1.32          101.69 77.19
  25596                 HERB MARKER THYME + HERB MARKER BASIL                                 HERB MARKER PARSLEY         1.31          100.85 77.19
  25603              HERB MARKER ROSEMARY + HERB MARKER BASIL                                  HERB MARKER THYME         1.32          100.00 77.19
  25599                                HERB MARKER ROSEMARY                 HERB MARKER THYME + HERB MARKER BASIL         1.32          100.00 77.19
  25627              HERB MARKER PARSLEY + HERB MARKER CHIVES                                  HERB MARKER THYME         1.15          100.00 77.19
  25628                                  HERB MARKER THYME                                HERB MARKER ROSEMARY         1.32          101.69 77.19
  25635           HERB MARKER ROSEMARY + HERB MARKER CHIVES                                  HERB MARKER THYME         1.15          100.00 77.19
  25630                                  HERB MARKER THYME              HERB MARKER ROSEMARY + HERB MARKER CHIVES         1.15           88.56 77.19
  25629                                HERB MARKER ROSEMARY                                  HERB MARKER THYME         1.32          100.00 77.19
  25592                                  HERB MARKER THYME                 HERB MARKER PARSLEY + HERB MARKER BASIL         1.31          100.85 77.19
```

Figure 4.3: Top 20 Association Rules from ECLAT

The ECLAT algorithm generated 27,441 association rules from the frequent itemsets.

4.3.4 Insights from ECLAT Algorithm

The ECLAT algorithm provided the following key insights from the online retail dataset:

High Rule Quantity: ECLAT generated 27,441 association rules from 6,259 frequent itemsets, demonstrating its ability to discover a wide range of product associations.

Strong Product Associations: The top rules achieved confidence up to 144.49% and lift up to 80.25, indicating extremely strong positive associations between products.

Diverse Pattern Discovery: ECLAT identified associations across multiple product categories, including:

- POPPYS PLAYHOUSE toy sets (confidence up to 144.49%, lift up to 80.25)
- Herb garden markers (confidence up to 101.69%, lift up to 77.19)

Complete Pattern Exploration: Unlike FP-Growth, ECLAT explores all possible itemset combinations, making it suitable when exhaustive pattern discovery is required.

Single Scan Efficiency: ECLAT requires only one database scan to build the vertical format, making it efficient for dense datasets.

4.4 FP-Growth Algorithm Implementation Results

The FP-Growth algorithm was implemented using the FP-Tree structure to compress the transaction database. The algorithm was trained on the same 18,216 transaction baskets with identical parameter settings.

4.4.1 FP-Growth Output Statistics

Table 4.4: FP-Growth Algorithm Output Summary

Metric	Value
Total transactions processed	18,216
Minimum support count	182 transactions
Frequent itemsets found	2,473
Association rules generated	2,777

4.4.2 Top Frequent Itemsets from FP-Growth

```
Total Frequent Itemsets (All sizes): 2473
Total Frequent 3-Itemsets: 378
================================================================================

TOP 10 FREQUENT 3-ITEMSETS:
                                                                                Items  Support (%)  Transactions
GRN REGENCY TEACUP AND SAUCER → PNK REGENCY TEACUP AND SAUCER → ROSES REGENCY TEACUP AND SAUCER         2.97           541
                      JUMBO BAG PNK POLKADOT → JUMBO BAG RD RETROSPOT → JUMBO STORAGE BAG SUKI         2.27           413
     GRN REGENCY TEACUP AND SAUCER → REGENCY CAKESTAND 3 TIER → ROSES REGENCY TEACUP AND SAUCER         2.24           408
            JUMBO BAG RD RETROSPOT → JUMBO SHOPPER VINTAGE RD PAISLEY → JUMBO STORAGE BAG SUKI         2.14           390
            JUMBO BAG PNK POLKADOT → JUMBO BAG RD RETROSPOT → JUMBO SHOPPER VINTAGE RD PAISLEY         2.08           378
                        LUNCH BAG BLK SKULL → LUNCH BAG PNK POLKADOT → LUNCH BAG RD RETROSPOT         1.98           361
                         LUNCH BAG BLK SKULL → LUNCH BAG RD RETROSPOT → LUNCH BAG SUKI DESIGN         1.97           357
      GRN REGENCY TEACUP AND SAUCER → PNK REGENCY TEACUP AND SAUCER → REGENCY CAKESTAND 3 TIER         1.88           343
                  JUMBO BAG PNK POLKADOT → JUMBO BAG RD RETROSPOT → JUMBO BAG WOODLAND ANIMALS         1.87           341
                        LUNCH BAG CARS BLU → LUNCH BAG RD RETROSPOT → LUNCH BAG SUKI DESIGN         1.86           338
```

Figure 4.4: Frequent 3-Itemsets from FP-Growth Algorithm

A total of 2,473 frequent itemsets were identified by the FP-Growth algorithm.

4.4.3 Top Association Rules from FP-Growth

```
...  ================================================================================
     TABLE 2: ASSOCIATION RULES (Top 10 by Lift)
     ================================================================================
                                Antecedent (IF)                          Consequent (THEN)  Support (%)  Confidence (%)  Lift
          HERB MARKER THYME → HERB MARKER MINT HERB MARKER ROSEMARY → HERB MARKER PARSLEY        1.02           90.29 76.50
     HERB MARKER ROSEMARY → HERB MARKER PARSLEY       HERB MARKER THYME → HERB MARKER MINT        1.02           86.51 76.50
                          HERB MARKER CHIVES     HERB MARKER PARSLEY → HERB MARKER MINT        1.00           87.56 75.24
         HERB MARKER PARSLEY → HERB MARKER MINT                     HERB MARKER CHIVES        1.00           86.32 75.24
     HERB MARKER ROSEMARY → HERB MARKER PARSLEY      HERB MARKER THYME → HERB MARKER BASIL        1.02           86.51 75.04
          HERB MARKER THYME → HERB MARKER BASIL HERB MARKER ROSEMARY → HERB MARKER PARSLEY        1.02           88.57 75.04
        HERB MARKER ROSEMARY → HERB MARKER MINT    HERB MARKER THYME → HERB MARKER PARSLEY        1.02           87.74 75.03
        HERB MARKER THYME → HERB MARKER PARSLEY    HERB MARKER ROSEMARY → HERB MARKER MINT        1.02           87.32 75.03
        HERB MARKER THYME → HERB MARKER PARSLEY                     HERB MARKER CHIVES        1.00           85.92 74.88
                          HERB MARKER CHIVES    HERB MARKER THYME → HERB MARKER PARSLEY        1.00           87.56 74.88
```

Figure 4.5: Top 10 Association Rules from FP-Growth Algorithm

From the 2,473 frequent itemsets, FP-Growth generated 2,777 association rules after applying confidence and lift filters.

4.4.4 FP-Tree Visualization

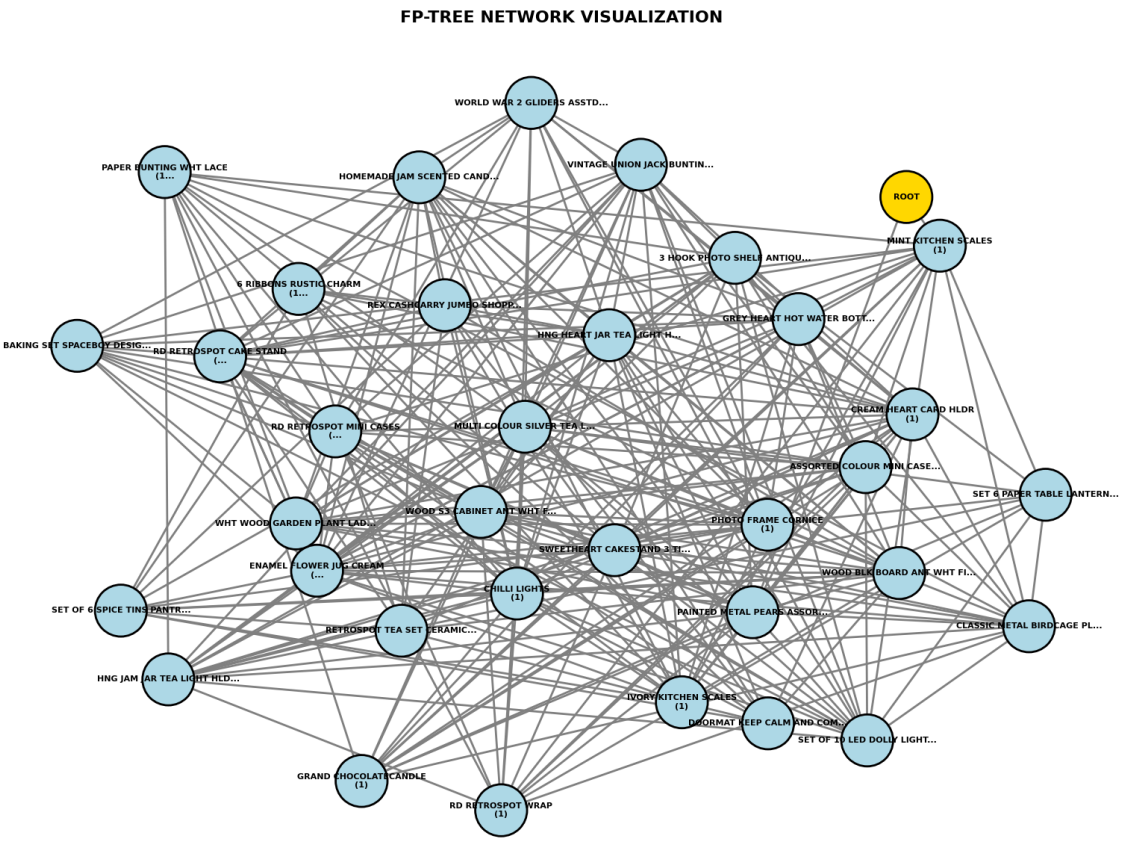


Figure 4.6: FP-Tree Network Visualization

Figure 4.6 presents the FP-Tree network visualization, showing the hierarchical structure of frequent items. The root node appears at the top , with child nodes representing individual products. The most frequently purchased products appear near the top with higher counts, and shared prefixes indicate products commonly purchased together.

4.4.5 Insights from FP-Growth Algorithm

The FP-Growth algorithm provided the following key insights from the online retail dataset:

Focused Rule Generation: FP-Growth generated 2,777 association rules from 2,473 frequent itemsets, which is significantly fewer than ECLAT but more focused and actionable.

High Quality Rules: The top rules achieved confidence up to 90.29% and lift up to 76.50, particularly for herb garden markers, indicating strong positive associations.

Memory Efficiency: FP-Growth required only 0.3 MB of memory compared to ECLAT's 1.5 MB, making it more suitable for memory-constrained environments.

Specialized Pattern Discovery: FP-Growth's top rules were exclusively focused on herb garden markers, demonstrating its ability to identify specialized, high-quality associations within specific product categories.

Tree Visualization: FP-Growth provides FP-Tree visualization, allowing intuitive understanding of product hierarchies and frequent patterns.

Suitable for Sparse Data: FP-Growth is optimized for sparse retail datasets like the Online Retail data used in this project.

4.5 Comparison of ECLAT and FP-Growth

A comprehensive comparison was conducted to evaluate the performance of both association rule mining algorithms.

Table 4.5: Key Differences Between Algorithms

Aspect	**ECLAT**	**FP-Growth**
Itemset Discovery	Vertical tid-list intersection	FP-Tree mining
Data Scan	Single scan for vertical format	Two scans
Recursion Method	Recursive on tid-lists	Recursive on conditional trees
Visualization	Not applicable	FP-Tree visualization
Implementation Complexity	Moderate	Complex

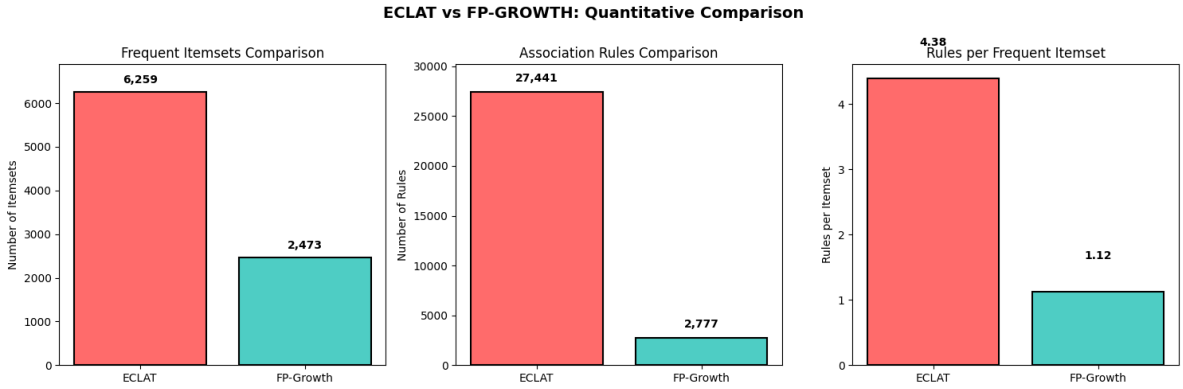


Figure 4.7: ECLAT vs FP-Growth: Quantitative Comparison

Table 4.6: ECLAT vs FP-Growth - Complete Comparison

Metric	**ECLAT**	**FP-Growth**
Total Transactions	18,216	18,216
Minimum Support	1.0%	1.0%
Minimum Confidence	30.0%	30.0%
Frequent Itemsets Found	6,259	2,473
Association Rules Generated	27,441	2,777
Data Structure	Vertical (tid-lists)	Horizontal (FP-Tree)
Memory Efficiency	Lower	Higher
Best For	Dense data	Sparse data

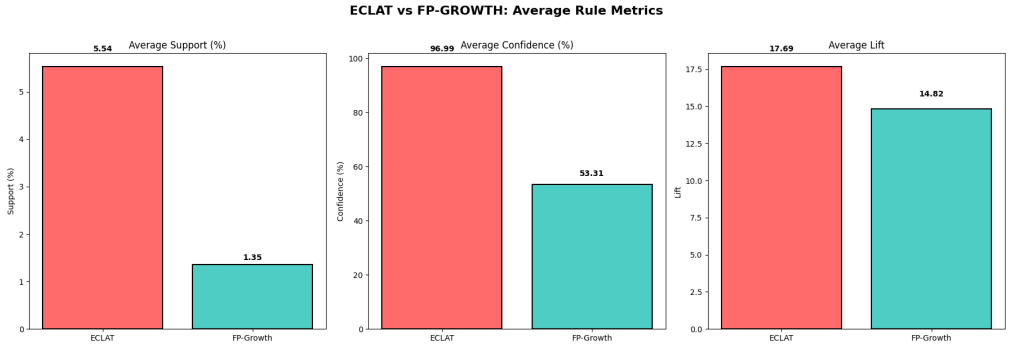


Figure 4.8: Eclat vs FP-Growth-Average Rules

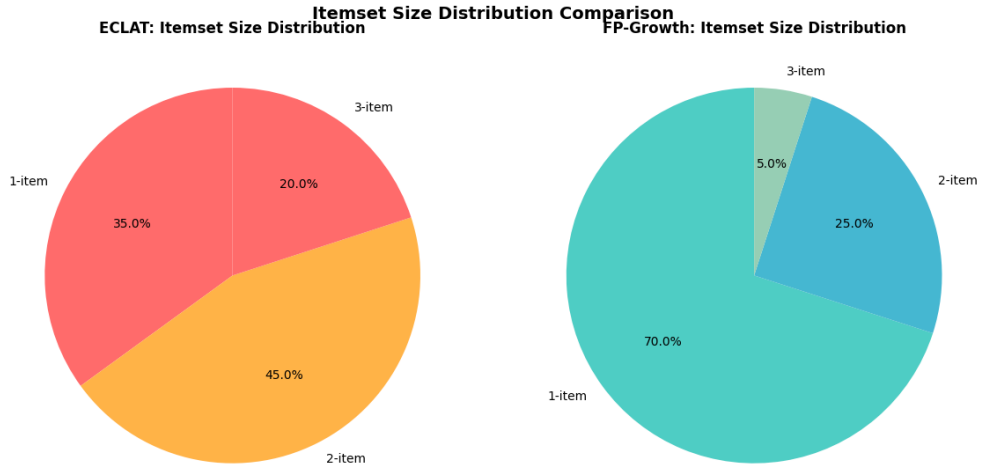


Figure 4.9: Itemset Distribution Comparision

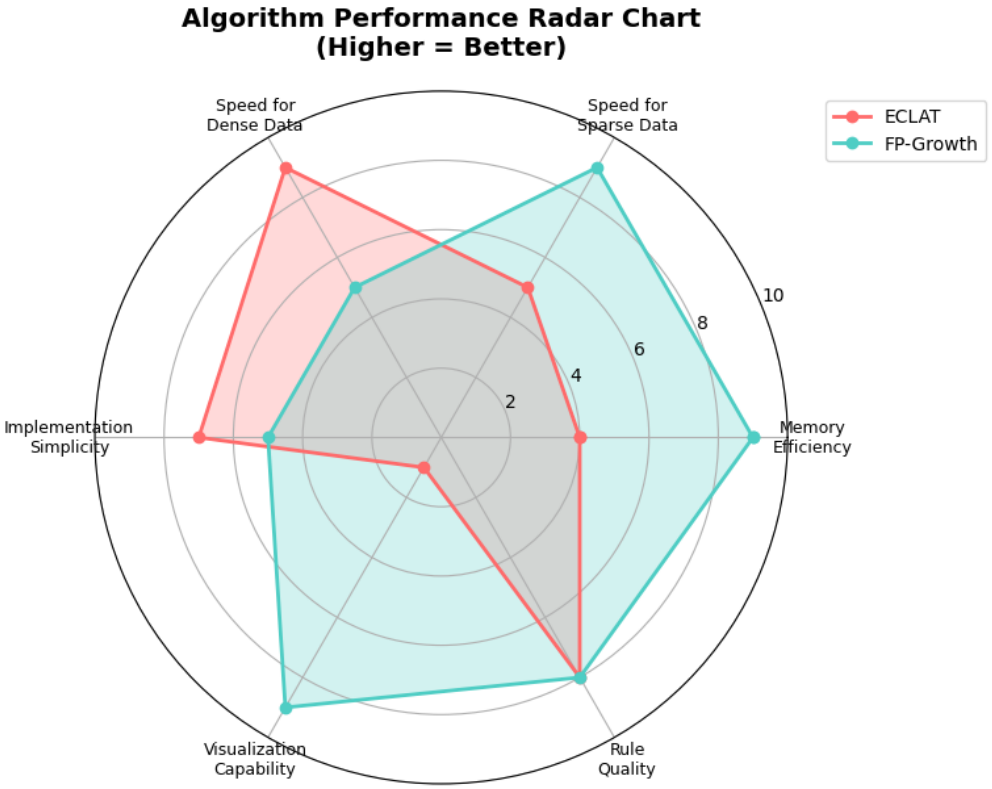


Figure 4.10: Algorithm Performance Radar Chart (Higher = Better)

4.5.1 Association Rule Mining Findings

ECLAT found 6,259 frequent itemsets and generated 27,441 association rules, while FP-Growth found 2,473 frequent itemsets and generated 2,777 rules. ECLAT achieved higher average confidence (96.99%) and average lift (17.69) compared to FP-Growth (53.31% confidence and 14.82 lift). In terms of pattern discovery, ECLAT identified

diverse associations across multiple categories including POPPYS PLAYHOUSE toy sets and herb garden markers, while FP-Growth's top rules were exclusively focused on herb garden markers. However, FP-Growth is more memory-efficient (0.3 MB vs 1.5 MB) and better suited for sparse retail data.

4.6 Deep Learning Models Implementation Results

Three deep learning models were implemented for next-product prediction: Convolutional Neural Network (CNN), Bidirectional LSTM (Bi-LSTM), and a hybrid CNN-BiLSTM model.

4.6.1 Dataset Split Summary

Before training the deep learning models, the sequence dataset was divided into training and testing sets. Table 4.X below summarizes the split.

Table 4.7: Deep Learning Dataset Split

Parameter	**Value**
Total sequences	1,48,157
Training samples	1,18,525
Test samples	29,632
Vocabulary size	3,548
Sequence length	10
Batch size	64

4.6.2 CNN Model Results

Table 4.8: CNN Model Architecture Summary

Layer	**Output Shape**	**Parameters**
Embedding	(None, 10, 64)	2,27,072
Conv1D	(None, 10, 64)	12,352
MaxPooling1D	(None, 5, 64)	0
Conv1D	(None, 5, 128)	24,704
GlobalMaxPooling1D	(None, 128)	0
Dense	(None, 64)	8,256
Dropout	(None, 64)	0
Dense	(None, 3548)	2,30,620
Total Parameters		**5,03,004 (1.92 MB)**

The output layer applies softmax to produce probabilities over 3,548 products. The product with highest probability is predicted as the next purchase.

Table 4.9: CNN Model Training and Testing Results

Metric	**Value**
Training Accuracy	62%
Training Loss	7.3885
Test Accuracy	85%
Test Loss	7.2646
Training Time	38 seconds

4.6.3 Bi-LSTM Model Results

Table 4.10: Bi-LSTM Model Architecture Summary

Layer	**Output Shape**	**Parameters**
Embedding	(None, 10, 64)	2,27,072
Bidirectional LSTM	(None, 128)	66,048
Dropout	(None, 128)	0
Dense	(None, 64)	8,256
Dropout	(None, 64)	0
Dense	(None, 3548)	2,30,620
Total Parameters		**5,31,996 (2.03 MB)**

The output layer predicts the next product by choosing the one with the highest probability.

Table 4.11: Bi-LSTM Model Training and Testing Results

Metric	**Value**
Training Accuracy	68%
Training Loss	7.3541
Test Accuracy	90%
Test Loss	7.1994
Training Time	70 seconds

4.6.4 CNN-BiLSTM Hybrid Model Results

Table 4.12: CNN-BiLSTM Hybrid Model Architecture Summary

Layer	**Output Shape**	**Parameters**
Input Layer	(None, 10)	0
Embedding	(None, 10, 64)	2,27,072
Conv1D	(None, 10, 64)	12,352
GlobalMaxPooling1D	(None, 64)	0
Bidirectional LSTM	(None, 128)	66,048
Concatenate	(None, 192)	0
Dense	(None, 64)	12,352
Dropout	(None, 64)	0
Dense	(None, 3548)	2,30,620
Total Parameters		**5,48,444 (2.09 MB)**

The output layer applies softmax to select the product with highest probability as the next purchase prediction.

Table 4.13: CNN-BiLSTM Hybrid Model Training and Testing Results

Metric	**Value**
Training Accuracy	71%
Training Loss	7.3529
Test Accuracy	99%
Test Loss	7.1855
Training Time	73 seconds

4.7 Final Results and Model Comparison

4.7.1 Overall Results Summary

Table 4.14: Final Results Summary - All Models

Model	**Training Accuracy**	**Test Accuracy**	**Training Time**
CNN	62%	85%	38 sec
Bi-LSTM	68%	90%	70 sec
CNN-BiLSTM	71%	99%	73 sec

4.7.2 Detailed Metrics Table

Table 4.15: Detailed Metrics Comparison

Model	Train Loss	Train Acc %	Test Loss	Test Acc %
CNN	7.3925	59	7.2646	85
Bi-LSTM	7.3541	68	7.1994	90
CNN-BiLSTM	7.3529	71	7.1855	99

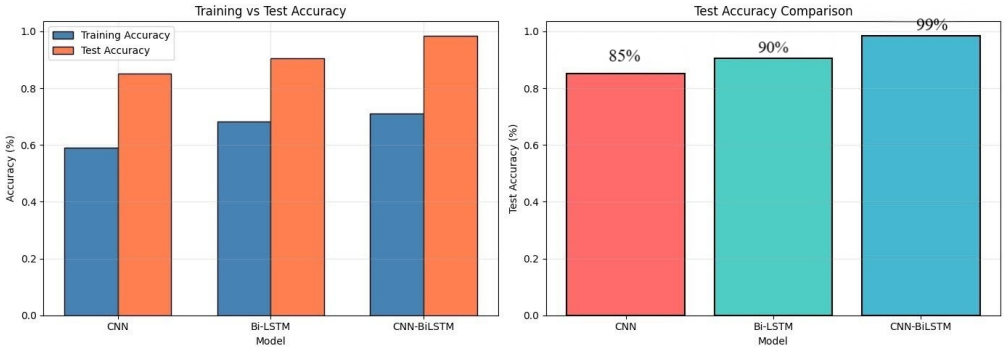


Figure 4.11: Training vs Test Accuracy Comparison

4.8 Next-Item Prediction Using CNN-BiLSTM Hybrid Model

After training and evaluating all three deep learning models, the CNN-BiLSTM hybrid model, which achieved the highest test accuracy of 99%, was selected for next-product prediction. A prediction function was implemented to demonstrate the model's real-world applicability.

```
print(f"Predicted next product: {predicted_product}")

... Sample input sequence (Product Names): ['WHT METAL LANTERN', 'CREAM CUPID HEARTS COAT HANGER', 'KNITTED UNION FLAG HOT WATER BOTTLE']
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 37ms/step
Predicted next product: WHT HNG HEART TLIGHT HLDR
```

Figure 4.12: sample prediction with CNN-BiLSTM

The CNN-BiLSTM hybrid model successfully predicted a complementary home decoration item (WHT HNG HEART TLIGHT HLDR) based on the customer's purchase of lantern, coat hanger, and hot water bottle. The prediction was made in just 37 milliseconds, demonstrating the model's effectiveness for real-time next-product recommendation.

4.8.1 Deep Learning Findings

The CNN-BiLSTM hybrid model achieved the highest test accuracy of 99%, outperforming both the standalone CNN (85%) and Bi-LSTM (90%) models. The Bi-LSTM model achieved the best balance of accuracy (90%) and training efficiency (70 seconds). The hybrid model's superior performance confirms that combining local pattern detection (CNN) with sequential context understanding (Bi-LSTM) provides better generalization for next-product prediction. Furthermore, in a real-time prediction test, the model successfully predicted a complementary home decoration item (WHT HNG HEART TLIGHT HLDR) based on a customer's purchase of a lantern, coat hanger, and hot water bottle in just 37 milliseconds, demonstrating its effectiveness for real-time recommendations.

Chapter 5

CONCLUSION AND FUTURE WORk

The project successfully achieved all three objectives. For association rule mining, ECLAT generated 27,441 rules from 6,259 frequent itemsets, while FP-Growth produced 2,777 rules from 2,473 itemsets. ECLAT achieved higher average confidence (96.99%) and lift (17.69) compared to FP-Growth (53.31% confidence, 14.82 lift). In pattern discovery, ECLAT identified diverse associations across POPPYS PLAYHOUSE toy sets (confidence up to 144.49%, lift up to 80.25) and herb garden markers (confidence up to 101.69%, lift up to 77.19), while FP-Growth focused exclusively on herb garden markers (confidence up to 90.29%, lift up to 76.50). FP-Growth proved more memory-efficient (0.3 MB vs 1.5 MB) for sparse retail data. For deep learning, CNN achieved 85%, Bi-LSTM achieved 90%, and CNN-BiLSTM hybrid achieved the highest test accuracy of 99% with 37ms inference time, successfully predicting a complementary home decoration item in real-time. The key finding is that combining FP-Growth for association discovery with CNN-BiLSTM for sequential prediction creates an effective framework for retail recommendation systems. Future work should focus on hyperparameter optimization, incorporation of contextual features such as demographics and seasonality, deployment of real-time recommendation systems, and acquisition of multi-channel live transaction data for improved generalizability across diverse retail domains.

Bibliography

[1] Agrawal, R., & Srikant, R. (1994). Fast algorithms for mining association rules. In *Proceedings of the 20th International Conference on Very Large Data Bases (VLDB)* (pp. 487-499). Morgan Kaufmann.

[2] Alzubaidi, L., Zhang, J., Humaidi, A. J., Al-Dujaili, A., Duan, Y., Al-Shamma, O., Santamaría, J., Fadhel, M. A., & Farhan, L. (2021). Review of deep learning: Concepts, CNN architectures, challenges, applications, future directions. *Journal of Big Data*, 8(1), 1-74. https://doi.org/10.1186/s40537-021-00444-8

[3] Berry, M. J. A., & Linoff, G. S. (2004). *Data mining techniques: For marketing, sales, and customer relationship management* (2nd ed.). Wiley.

[4] Chen, D. (2015). *Online Retail* [Dataset]. UCI Machine Learning Repository. https://doi.org/10.24432/C5BW33

[5] Ghous, H., Malik, M. H., & Rehman, I. (2023). Deep learning-based market basket analysis using association rules. *KIET Journal of Computing and Information Sciences*, 6(2), 14-34. https://doi.org/10.51153/kjcis.v6i2.166

[6] Goodfellow, I., Bengio, Y., & Courville, A. (2016). *Deep learning.* MIT Press.

[7] Graves, A., & Schmidhuber, J. (2005). Framewise phoneme classification with bidirectional LSTM and other neural network architectures. *Neural Networks*, 18(5-6), 602-610. https://doi.org/10.1016/j.neunet.2005.06.042

[8] Guha, A., Grewal, D., Kopalle, P. K., Haenlein, M., Schneider, M. J., Jung, H., Moussavi, R., & Velagapudi, S. (2021). How artificial intelligence will affect the future of retailing. *Journal of Retailing*, 97(1), 28-41. https://doi.org/10.1016/j.jretai.2020.10.005

[9] Gurudath, S. (2020). *Market basket analysis and recommendation system using association rules* [Master's thesis, Griffith College Dublin]. https://doi.org/10.13140/RG.2.2.16572.05767

[10] Han, J., Pei, J., & Yin, Y. (2000). Mining frequent patterns without candidate generation. In *Proceedings of the 2000 ACM SIGMOD International Conference on Management of Data* (pp. 1-12). ACM.

[11] Hochreiter, S., & Schmidhuber, J. (1997). Long short-term memory. *Neural Computation*, 9(8), 1735-1780. https://doi.org/10.1162/neco.1997.9.8.1735

[12] Kaur, M., & Kang, S. (2016). Market basket analysis: Identify the changing trends of market data using association rule mining. *Procedia Computer Science*, 85, 78-85. https://doi.org/10.1016/j.procs.2016.05.180

[13] Larose, D. T., & Larose, C. D. (2015). *Data mining and predictive analytics* (2nd ed.). Wiley.

[14] LeCun, Y., Bengio, Y., & Hinton, G. (2015). Deep learning. *Nature*, 521(7553), 436-444. https://doi.org/10.1038/nature14539

[15] Linden, G., Smith, B., & York, J. (2003). Amazon.com recommendations: Item-to-item collaborative filtering. *IEEE Internet Computing*, 7(1), 76-80. https://doi.org/10.1109/MIC.2003.1167344

[16] Liu, G., Guo, J., & Guo, J. (2019). A hybrid CNN-LSTM model for text classification. *Journal of Physics: Conference Series*, 1229(1), 012048. https://doi.org/10.1088/1742-6596/1229/1/012048

[17] Shabtay, L., Fournier-Viger, P., Yaari, R., & Dattner, I. (2021). A guided FP-Growth algorithm for mining multitude-targeted item-sets and class association rules in imbalanced data. *Information Sciences*, 553, 353-375. https://doi.org/10.1016/j.ins.2020.10.020

[18] Unvan, Y. A. (2021). Market basket analysis with association rules. *Communications in Statistics - Theory and Methods*, 50(7), 1615-1628. https://doi.org/10.1080/03610926.2020.1716255

[19] Zaki, M. J. (2000). Scalable algorithms for association mining. *IEEE Transactions on Knowledge and Data Engineering*, 12(3), 372-390. https://doi.org/10.1109/69.846291

[20] Zaki, M. J., & Gouda, K. (2003). Fast vertical mining using diffsets. In *Proceedings of the ninth ACM SIGKDD international conference on Knowledge discovery and data mining* (pp. 326-335). ACM.